

Dipan Design OS

A personal operating system for clear, credible and sophisticated product interfaces

Prepared for Dipan Nama

September 2026

- [Dipan Design OS](#)
 - [A personal operating system for starting and designing future products](#)
 - [How to use this document](#)
- [Part I - Evidence, taste and judgment](#)
 - [1. Purpose, ownership and boundaries](#)
 - [2. Evidence and method](#)
 - [3. Dipan's taste profile](#)
 - [4. What modern means](#)
 - [5. Product truth before visual style](#)
 - [6. The quality hierarchy](#)
 - [7. Taste as trained judgment](#)
 - [8. Originality: borrow grammar, not identity](#)
- [Part II - Foundations](#)
 - [9. Semantic token architecture](#)
 - [10. Color systems and accent budgets](#)
 - [11. Typography roles](#)
 - [12. Font selection](#)
 - [13. Spacing and rhythm](#)
 - [14. Grid and container logic](#)
 - [15. Radius, borders and nested geometry](#)
 - [16. Depth, shadows, blur and glass](#)
 - [17. Iconography](#)
 - [18. Illustration and art direction](#)
 - [19. Content design](#)
- [Part III - Page construction and product storytelling](#)
 - [20. Landing-page narrative architecture](#)
 - [21. Type-led heroes](#)
 - [22. Product proof as evidence](#)
 - [23. Ratio cadence](#)
 - [24. Quiet framing](#)

- 25. Section rhythm and density
- 26. Bento as a local tool
- 27. Split sections
- 28. Comparison, trust and objection handling
- 29. Final calls to action
- 30. Mobile composition
- 31. SaaS and product OS pattern
- 32. Developer-tool and infrastructure pattern
- 33. Desktop app and command surface pattern
- 34. Editorial, course and journal pattern
- 35. Component library and docs pattern
- 36. Event and conversion pattern
- 37. Findings from all 245 dark.design listings
- Part IV - Component architecture and the shadcn ecosystem
 - 38. shadcn/ui is open code
 - 39. Primitive to page
 - 40. State completeness
 - 41. Buttons and actions
 - 42. Forms and validation
 - 43. Navigation
 - 44. Dialogs, drawers, popovers and tooltips
 - 45. Tables, lists and data density
 - 46. Feedback and perceived performance
 - 47. Empty, loading and error states
 - 48. Registry and library landscape
 - 49. Free template research
 - 50. Product showcase study method
- Part V - Motion and micro-interaction
 - 51. Motion decision framework
 - 52. Timing and easing
 - 53. CSS transitions and keyframes
 - 54. Motion for React
 - 55. Enter, exit and presence
 - 56. Layout and shared elements
 - 57. Gestures and direct manipulation
 - 58. Micro-interaction anatomy
 - 59. Component motion
 - 60. Reduced motion
 - 61. Performance

- 62. Emil Kowalski and interfaces.dev
- Part VI - Accessibility, responsiveness and quality
 - 63. Semantic HTML
 - 64. Keyboard and focus
 - 65. Contrast and non-color meaning
 - 66. Target size and touch
 - 67. Form errors
 - 68. Responsive systems
 - 69. Container queries
 - 70. Responsive typography
 - 71. Images and performance
 - 72. Loading and perceived speed
 - 73. Visual QA
 - 74. Technical QA
- Part VII - Dipan's operating system
 - 75. Project kickoff worksheet
 - 76. Reference brief
 - 77. Visual thesis
 - 78. Page recipe selector
 - 79. Implementation sequence
 - 80. Review rubric
 - 81. Anti-patterns
 - 82. Master checklist
 - 83. Free-resource policy
 - 84. Dipan's expanded inspiration map
 - 85. Durable principles versus trends
 - 86. Maintaining the Design OS
- Appendix A - Representative chapter
- Representative chapter sample: Product imagery is evidence, not decoration
 - Start by assigning a job to every visual
 - What the 245-site audit says
 - Dipan's ratio cadence
 - Framing rules
 - Crop for the claim
 - Motion belongs to the change
 - Implementation pattern
 - Review checklist
 - Decision rule

- [Appendix B - Source ledger](#)
- [Source ledger - checkpoint 1](#)
 - [User and project evidence](#)
 - [Full collection dataset](#)
 - [Component model and free ecosystems](#)
 - [Design engineering, taste, and motion](#)
 - [Accessibility and responsive quality](#)
 - [Typography and icons](#)
 - [User-supplied research expansion - September 19, 2026](#)
 - [87. Representative gallery sampling: what the current collections actually reward](#)
 - [88. Durable principles versus visual trends](#)
 - [89. Gallery research protocol for future products](#)

Dipan Design OS

A personal operating system for starting and designing future products

This document is Dipan's private design source of truth. It combines the lessons from fifteen interface studies, a complete audit of all 245 external listings in the dark.design gallery, current primary documentation for open-code component systems and motion, public design-engineering guidance, accessibility standards, and Dipan's direct feedback.

It is not a gallery of fashionable effects. It is a decision system: how to turn a product truth into information architecture, visual hierarchy, trustworthy proof, complete component states, responsive composition and production-quality interaction.

Core rule: Type establishes the promise. Imagery proves it. Interaction explains the change.

How to use this document

Use the report in three modes:

1. **Starting a project:** complete the brief, choose a visual thesis, establish foundations, select a page or product grammar, and plan proof before implementation.

2. **During implementation:** use the component, motion, accessibility and responsive chapters as working constraints.
3. **During review:** use the quality hierarchy, anti-patterns and release gates to identify the highest-value correction first.

Do not apply every pattern. The system is a menu governed by product fit. A developer tool, course, desktop app and component library should not look identical. They should share judgment quality.

Part I - Evidence, taste and judgment

1. Purpose, ownership and boundaries

This system belongs to Dipan. It records his preferences and working defaults, not universal aesthetic law. The goal is to reduce the cost of a blank page without reducing every future product to one template.

The system favors free and open resources. A resource appearing in the research does not automatically make it approved: verify its current license, maintenance, dependencies, accessibility model and free scope at the time of use. Open code is valuable because it can be inspected and adapted; it also transfers responsibility for what is shipped.

The design system is private by default. References may be public, but the synthesis, preference profile and project decision logic are personal. Any public showcase must exclude private product material and must be explicitly approved before publication.

The final authority is the rendered product. A clean component tree or passing CI run does not prove that type wraps well, a screenshot crop works on mobile, a focus ring is visible, or a dense page has rhythm. Visual and interaction QA are part of implementation, not presentation after implementation.

2. Evidence and method

The research uses four evidence classes.

Owner evidence is Dipan's direct feedback. It establishes personal preference: minimalist modern SaaS logic, section-specific composition, no repeated images, no coded imitation beside real proof, restrained color, negative space, type-led hierarchy, purposeful motion, mobile responsiveness and distinct page identities.

Internal project evidence is the set of fifteen design skills derived from actual studies: adaptive SaaS, command-center productivity, component storefront, desktop app, self-demonstrating tool, developer engine, technical journal, annotated editorial, illustration commerce, product OS, social publishing, developer infrastructure, docs, interactive course and event conversion.

Dataset evidence is the exact 245-list dark.design audit. It retains 211 inspectable listings and 34 unavailable or blocked listings instead of silently dropping failures. DOM signals and declared image ratios are treated as structural indicators with known limits.

Public primary evidence includes shadcn/ui, Motion, W3C, MDN, React Aria, official font and icon projects, interfaces.dev, and Emil Kowalski's published work. Secondary directories are useful for discovery but should not carry load-bearing claims without a primary source.

3. Dipan's taste profile

Dipan likes interfaces that look intentional rather than decorated. His preferred balance is minimalist but informative: a calm field, strong type, enough real product evidence to establish credibility, and interaction details that make the product feel considered.

Positive signals:

- one focal idea per section or viewport;
- a clear headline with controlled line breaks;
- one accent rather than a rainbow of emphasis;
- quiet dark surfaces or light editorial fields;
- finished screenshots cropped to the claim;
- a mix of overview, focused landscape and occasional portrait proof;
- subtle inset framing and hairline borders;
- local bento where comparison benefits from simultaneous visibility;
- dense proof followed by a sparse pause;
- motion used for state, hierarchy, continuity or explanation;
- real mobile restructuring;
- distinct products that still share a high quality bar.

Negative signals:

- one rounded card template repeated down the page;
- imagery as filler;
- unreadable full-app screenshots;
- CSS mock interfaces duplicating real screenshots;
- repeated image files or duplicated bytes;
- heavy accumulation of gradients, glow, pills, labels and badges;
- constant float, parallax or hover motion;
- desktop compressed into mobile;
- generic startup copy and invented proof;
- a design system that erases product character.

4. What modern means

Modern is not a list of current visual effects. It is a product that reflects current expectations: clear hierarchy, fast response, accessible interaction, fluid adaptation, credible proof, coherent states and a visual language that does not fight the task.

A design can use serif type, plain borders and no gradients and still feel modern if it has good content, state design and responsive behavior. A design can use glassmorphism, animated gradients and 3D cards and feel dated immediately if those effects are disconnected from the product.

Treat trends as optional vocabulary. Evaluate each one with four questions:

1. Does it clarify information or interaction?
2. Does it support the product's voice?
3. Does it survive mobile, accessibility and performance constraints?
4. Will the interface still work when the effect is removed?

A current product should feel native to its medium, not eager to prove that it has seen the same inspiration boards as everyone else.

5. Product truth before visual style

Every interface starts with six facts:

- **User:** who is here?
- **Situation:** what are they trying to do now?
- **Pain:** what friction or risk exists?
- **Promise:** what gets better?
- **Proof:** what evidence can establish that claim?
- **Action:** what is the next useful step?

Write these before choosing a component library or visual theme. If the page cannot state its promise in one sentence, the design will compensate with noise. If the product cannot supply proof, build an honest process diagram, artifact sample or workflow demonstration. Do not invent metrics, logos or testimonials.

Good design narrows ambiguity. The primary CTA should follow naturally from the promise. Secondary actions should answer a real hesitation, such as viewing documentation, contacting sales or seeing an example, not merely provide symmetry.

6. The quality hierarchy

Review in this order:

1. **Clarity:** can the user understand the current page, task and next action?
2. **Coherence:** do type, spacing, color, imagery, states and interaction agree?
3. **Credibility:** is important copy supported by honest, legible evidence?
4. **Delight:** do craft details improve feel without slowing the user?

This order prevents polishing the wrong layer. A smooth card hover cannot fix an unclear information hierarchy. A refined palette cannot make a tiny screenshot credible. A clever transition cannot excuse a missing error state.

When a design feels wrong, diagnose the highest broken layer. Corrections become cheaper and more durable.

7. Taste as trained judgment

Emil Kowalski describes taste as trained instinct rather than personal preference in [Developing Taste](#). The practical implication is that taste can be improved through exposure, analysis and repetition.

Use this loop:

1. Surround yourself with strong, relevant work.
2. Use products, not only screenshots.
3. Name why something works: hierarchy, crop, timing, language, spacing, state or restraint.
4. Compare several solutions to the same problem.
5. Rebuild the principle with the current product's real content.
6. Remove a decorative layer and check whether clarity improves.

7. Render, critique and repeat.

Keep a decision log. “Feels premium” is not a usable explanation. “The muted borders let the selected row and destructive action keep the only strong contrast” can guide implementation and review.

8. Originality: borrow grammar, not identity

Reference research should extract problem-solving grammar:

- type-led hero before product proof;
- alternating dense and sparse sections;
- local capability grid;
- framed product crop;
- shared-element transition;
- persistent command surface.

Do not copy distinctive identity:

- proprietary copy or information sequence;
- brand illustrations or visual metaphors;
- signature gradients, mascots or motifs;
- a competitor’s product scene reconstructed with renamed labels;
- exact choreography that is strongly associated with one product.

For every borrowed pattern, record the problem it solves, why it fits this product, how real content changes the implementation, and which source traits must remain behind. This turns inspiration into reasoning.

Part II - Foundations

9. Semantic token architecture

Tokens should communicate purpose. Components should request `surface-raised`, `text-muted`, `border-interactive`, `focus-ring` and `danger`, not hard-coded neutral steps. Primitive scales still exist underneath, but semantic names let themes and product contexts evolve without rewriting every component.

Minimum color roles:

- canvas, surface, raised surface, inset surface;
- primary, secondary and muted text;
- subtle, default and strong borders;
- accent background and foreground;
- success, warning, danger and information pairs;
- focus and overlay.

Add semantic scales for spacing, radius, elevation, duration and easing. Keep the public token set smaller than the raw scale. Too many nearly identical values make inconsistency easy and review difficult.

Dark mode requires separate judgment. Do not invert light mode. Raised surfaces may need more luminance, borders may need lower alpha, and images may need an inset edge so they do not disappear into the canvas.

10. Color systems and accent budgets

Start neutral and earn color. One accent is usually sufficient for the primary action, active state, key data, focus and occasional emphasis. Semantic state colors are not decorative palette expansion.

Define an accent budget before building:

- primary CTA and current selection may use full accent;
- hover can use a restrained accent tint;
- focus needs strong visibility, even if that means a separate focus color;
- icons inherit neutral text unless they communicate state;
- borders do not become accent by default.

Contrast is an information requirement. Avoid reducing text contrast to make a page look expensive. Create subtlety with size, weight, position and spacing before reducing legibility.

In dark mode, pure black can create harsh edges and exaggerated glow. A softened near-black often provides more control, but the choice must maintain contrast. In light mode, off-white can reduce glare, but product screenshots and white surfaces still need clear boundaries.

11. Typography roles

Design type as roles, not a pile of sizes:

- display;
- page title;
- section title;
- lead;
- body;
- label;
- caption;
- code and data.

A role specifies family, weight, size range, line height, tracking, measure and behavior. Display type can use tight tracking and compact line height; body needs more air. Labels should remain readable instead of shrinking into decoration.

Use `text-wrap: balance` for short headings and `text-wrap: pretty` for short descriptions. Do not balance long-form reading text. Cap long-form measure around 60-75 characters. Use tabular numerals in changing values, tables, counters, prices and timers.

Use natural case in source copy and control transformation through style. Preserve smart punctuation and meaningful symbols. Underlines should clear descenders. Truncated values need a way to reveal the full value.

12. Font selection

Choose type for product voice and rendering behavior, not trend rank.

- **Inter** is a durable neutral product UI family with broad character and language support.
- **Geist** is compact and technical, fitting developer tools and precise SaaS.
- **Plus Jakarta Sans** has friendlier geometry for consumer and marketing contexts.
- **Manrope** is open and geometric with a warmer personality.

Use one family plus a mono by default. A second display family is justified only when it creates a meaningful editorial identity. Variable fonts reduce file count and enable optical or weight nuance, but still require subset and loading discipline.

Use WOFF2 on the web and self-host where the project needs predictable privacy, performance or control. Verify licenses from official sources. Test real product strings, numerals, punctuation, code, uppercase labels and multilingual content. A beautiful specimen page does not reveal whether the family supports a dense table or Indian names cleanly.

13. Spacing and rhythm

Use a 4px primitive grid, then a semantic scale. Common component gaps often fit 4, 8, 12, 16, 24 and 32. Page rhythm can extend through 48, 64, 96 and 128. The exact scale matters less than consistent relationships.

Define:

- narrow reading width;
- standard content container;

- wide proof container;
- full-bleed exception;
- mobile side padding;
- compact, standard and spacious section gaps.

Spacing communicates grouping. Elements inside a component should be closer to one another than to adjacent components. A heading and its paragraph form one group; the next proof surface begins after a larger gap.

Optical alignment can override geometry. Icons, curved forms and mixed-weight type may need small corrections. Make those corrections explicit rather than growing the gap until the misalignment is harder to see.

14. Grid and container logic

A grid is a coordination system, not a visible cage. Use it to align type, imagery, controls and section transitions. Let exceptions be deliberate.

A useful landing-page system can include:

- 4 columns on mobile;
- 8 on tablet;
- 12 on desktop;
- stable outer margins;
- a standard content width and a wider product-proof width.

Do not force every section to the same span. A quote may occupy six columns, a product overview ten, and a full-bleed editorial image twelve. Shared edges create coherence; varied spans create rhythm.

Use CSS Grid when two-dimensional alignment matters, Flexbox for one-dimensional flow, and container queries when a component's behavior depends on its local width rather than the viewport. Avoid breakpoint proliferation. Content constraints should trigger change.

15. Radius, borders and nested geometry

Use a small radius family: compact controls, standard cards and large feature surfaces. Too many radii make a product feel assembled from unrelated kits.

Nested radii should be concentric. A practical starting point is $\text{inner radius} = \text{outer radius} - \text{inset}$, then optical correction. The [interfaces.dev cheat sheet](#) specifically calls out concentric nested radii and subtle inset image outlines.

Borders establish an edge. Shadows establish elevation. Applying both strongly creates heavy chrome. On dark surfaces, a low-alpha light border may be enough. On light surfaces, a gray border or subtle inset edge often separates white imagery without a floating shadow.

Pills are for states, compact categories and controls whose shape communicates containment. Turning every label and action into a pill removes hierarchy.

16. Depth, shadows, blur and glass

Depth should describe stacking or focus. A popover sits above content; a modal sits above an overlay; a dragged item lifts from its list. Shadows without a spatial reason become decoration.

Use a small elevation scale. Tune color and spread for each theme. Large soft shadows can suit a hero proof, but should not appear on every card. In dark mode, shadow alone may be invisible; surface luminance and border often carry the separation.

Blur and glass are expensive visually and technically. Use them for transient overlays, floating controls or a product voice that genuinely benefits. Always provide enough opaque fallback for contrast. Avoid stacking translucent layers until text and edges become muddy.

Glow belongs to a luminous signal: active agent, running process, live status or cinematic focal point. It should not be the default way to make a dark card “modern.”

17. Iconography

Use one primary family per product.

- [Lucide](#) fits neutral product UI and aligns naturally with shadcn/ui.
- [Phosphor](#) provides multiple weights and more personality.
- [Tabler Icons](#) offers a broad consistent outline set.
- [Radix Icons](#) are compact and effective in dense controls.
- [Remix Icon](#) provides broad interface coverage.

Normalize size, stroke and optical weight. Do not mix families casually. Icons support labels; unfamiliar actions still need text or a tooltip. Icon-only controls require an accessible name and enough target area. Decorative icons should be hidden from assistive technology.

Use brand icons only from reliable official sources and preserve their rules. Do not redraw a company mark from an approximate generic icon.

18. Illustration and art direction

Illustration is useful when a product concept cannot be shown directly, when an editorial pause benefits from atmosphere, or when a product has a strong maker identity. It should have a repeatable visual grammar: stroke, shape, palette, perspective, texture and character treatment.

Avoid generic blob art. If the illustration cannot be tied to a concept, user situation or product outcome, it is decoration. For product education, prefer diagrams that preserve relationships. For emotional storytelling, a photograph or commissioned-style artwork can carry mood, but it must not pretend to be product proof.

Generated imagery requires the same art direction and originality rules as any other source. Inspect hands, text, perspective, UI plausibility, cultural signals and visual consistency. Record source and rights.

19. Content design

Interface copy is part of the component API. Buttons state the action, errors state the problem and recovery, empty states state what is absent and how to proceed, and labels use the user's language.

Prefer plain words. Remove generic claims such as “streamline your workflow” unless the next sentence explains exactly what changes. A landing-page section should usually contain one claim, one explanation and one piece of evidence.

Use sentence case. Keep labels stable across surfaces. Avoid using several synonyms for the same object. Use progressive disclosure for detail rather than dense preambles.

A confident product voice does not need constant exclamation, inflated adjectives or badge clutter. Specificity creates credibility.

Part III - Page construction and product storytelling

20. Landing-page narrative architecture

A landing page should create a sequence of belief:

1. **Recognition:** this is relevant to me.
2. **Promise:** this changes something I care about.
3. **Orientation:** I understand what the product is.
4. **Proof:** I can see how it works or what it produces.
5. **Fit:** it works with my context, tools or constraints.
6. **Trust:** the people, product and terms are credible.
7. **Action:** the next step is clear and proportionate.

The order changes by product. A developer tool may lead with concrete code. A new consumer product may need emotional context before mechanism. A desktop utility may benefit from an immediate app image. An enterprise product may need category and trust before details.

Assign every section one dominant role: promise, orientation, proof, explanation, comparison, trust, objection handling or action. If a section tries to do several at equal strength, split it.

21. Type-led heroes

A hero is not a summary of the entire website. Its job is to establish relevance, promise and next step. Use one clear headline, a supporting sentence, one primary action and at most one meaningful secondary action.

Type-led heroes are effective when the product category is understandable and a large proof surface follows. They create focus and let product evidence arrive at useful scale. Avoid decorating the empty space simply because the hero has no image.

A hero image is justified when the product itself is unfamiliar, the device or visual result is the promise, or an emotional scene is central to positioning. Even then, the image must remain subordinate to comprehension.

Control line breaks intentionally at target widths. Headline wrapping is part of composition. Do not use manual `<br>` without testing localization and mobile. A max-width plus balanced wrapping often gives better resilience.

22. Product proof as evidence

A product image must answer a question. Common roles:

Role	Question	Typical treatment
Overview	What is this?	16:9 or ultrawide product frame
Feature proof	Can it do this?	Focused 3:2 or 4:3 crop
Outcome proof	What changed?	Result, chart, artifact or before/after
Process	How does it work?	Diagram, timeline or controlled motion
Editorial pause	How should this feel?	Portrait art, photograph, quote or type
Comparison	How are these capabilities different?	Local grid or bento

If a visual answers nothing the copy cannot, remove it.

A screenshot should preserve orientation, action and consequence. Remove peripheral navigation, inactive tabs, empty canvas and repeated chrome before shrinking the action. The final rendered size, not source resolution, determines legibility.

23. Ratio cadence

The dark.design audit measured 2,768 images with declared ratios:

- 1,339 square-ish, 48.4%;
- 722 landscape, 26.1%;
- 388 ultrawide, 14.0%;
- 319 portrait, 11.5%.

The lesson is role-based variation, not statistical imitation.

Dipan's default cadence:

- hero proof: 16:9 or 16:10;
- focused workflow: 3:2 or 4:3;
- capability cluster: 1:1 or 5:4;
- portrait interruption: 4:5 or 5:6;
- panorama/divider: 2:1 or wider only if content remains legible.

Use one dominant ratio, one supporting ratio and an occasional interruption. Random variation feels as templated as strict uniformity.

24. Quiet framing

A good frame clarifies an edge and integrates the proof with the page.

Use:

- one outer surface;
- one hairline border;
- one inner radius;
- a restrained shadow only when elevation matters;

- visible inset space where editorial breathing room helps.

Use a browser bar only when browser context matters. Use a device frame only when device behavior matters. Avoid fake window controls as generic decoration.

Never show a coded fake of the same product scene beside, above or below a real image. The imitation competes with evidence and creates maintenance drift.

25. Section rhythm and density

Long pages remain readable by changing density and composition. A useful rhythm is:

- promise with negative space;
- large product proof;
- sparse principle or outcome row;
- split feature with focused proof;
- local comparison or portrait interruption;
- trust or update surface;
- quiet final action.

Do not repeat the sequence mechanically. The important pattern is contrast: dense then sparse, wide then narrow, image-led then type-led, overview then detail.

The audit classified 115 inspectable pages as high density, 52 as low and 44 as medium using text, section and media indicators. The number does not imply that high density is better. It shows that sophisticated pages often contain substantial content but use pacing to keep it navigable.

26. Bento as a local tool

Bento works when several capabilities should be visible at once and deserve unequal emphasis. It is especially useful for showing one large proof, two supporting capabilities and one compact metric or integration.

Keep bento local:

- three to six cells;
- varied spans based on information importance;
- different internal compositions where content requires it;
- one shared token and framing system;
- a return to simpler flow afterward.

Do not use bento because blank sections feel difficult. Explicit bento or masonry signals appeared on only 2.4% of inspectable dark.design sites, while grid alignment appeared on 61.1%. The broader pattern is disciplined alignment, not bento everywhere.

27. Split sections

A split section works when copy and proof explain one another. Use asymmetric columns when the proof needs more space. Alternate direction sparingly to create rhythm, not zig-zag for its own sake.

The copy side should contain a section label if useful, a clear claim, short explanation and optional deep link. The media side should show one focal operation. Do not turn the copy column into a feature list and the media column into a full dashboard simultaneously.

On mobile, stack based on comprehension. Copy usually precedes proof, but a familiar claim may benefit from showing the artifact first. Test the actual sequence instead of relying on DOM convenience.

28. Comparison, trust and objection handling

Comparison is useful when it helps a user choose or understand tradeoffs. Keep dimensions explicit and meaningful. Avoid a grid of green checks designed only to make one plan look inevitable.

Trust can come from:

- real customer evidence;
- security and reliability practices;
- transparent pricing or limitations;
- product change logs;
- team credibility;
- documentation quality;
- visible support and recovery paths.

Use logos only with permission and relevance. Testimonials need identity and context. Numbers need a source. A well-designed product walkthrough can be stronger proof than a row of unsupported brand marks.

FAQs should answer real objections not already addressed. Group them by topic. Keep the answer concise and link to deeper documentation where needed.

29. Final calls to action

The final CTA should restate the next step, not restart the whole pitch. Match commitment level to user readiness: “Try the demo,” “Create a workspace,” “Read the docs,” or “Talk to an engineer” can be more honest than “Get started.”

A final action surface can be visually distinct, but it should preserve the page’s type, radius and color system. Avoid introducing a new gradient, illustration and button style at the last moment.

Include practical reassurance near the action when true: no card, local-only, cancel anytime, free tier, setup time or platform support. Do not hide important terms in tiny muted text.

30. Mobile composition

Mobile is a priority system. For every desktop section decide:

- what remains visible;
- what reorders;
- what re-crops;
- what becomes progressive disclosure;
- what can disappear without losing meaning.

Product screenshots often need a new crop. Options:

1. re-crop around the active operation;
2. reorder claim then proof;
3. place secondary proof behind tabs, steps or a controlled carousel;
4. substitute a mobile-native or result view.

Keep touch targets generous, sticky controls away from safe-area conflicts, and headings within reasonable viewport depth. Do not preserve desktop side-by-side layouts as tiny columns. A mobile page should feel authored, not collapsed.

31. SaaS and product OS pattern

Recommended grammar:

- clear promise and fit;
- one large product overview;
- outcomes or principles;
- focused workflow proofs;
- collaboration/integration fit;
- trust and action.

Use UI imagery for claims about workflow, not as background texture. Product OS pages often need dense proof; follow it with a sparse statement. Show how objects relate: goal to project, task to result, person to agent, input to output.

Avoid generic three-card feature rows repeated several times. Make each major section answer a different product question.

32. Developer-tool and infrastructure pattern

Lead with specificity: what the tool accepts, produces, connects or improves. A code sample is evidence only when it is readable and plausible.

Useful sections:

- promise and minimal operational proof;
- install or first-success path;
- architecture or data flow;
- reliability/performance evidence;
- integrations and docs;
- clear developer CTA.

Dark surfaces suit this category but are not mandatory. Use accent for syntax, live state or success. Do not overload the page with terminal windows. One strong code surface plus a product result is usually more credible.

33. Desktop app and command surface pattern

Show the app at useful scale. A calm hero, download action and platform certainty often outperform a busy feature grid.

Focus on workflows, shortcuts, local or cloud behavior, privacy and platform fit. Keyboard-first products need command discoverability and a clear relationship between shortcut and result. Avoid animating frequent keyboard actions; speed is part of the promise.

Use full app overview once. Later sections should crop specific workflows rather than repeat the same window.

34. Editorial, course and journal pattern

Use type contrast, reading measure and illustration to create a publication rhythm. The page can be visually rich without card-heavy UI.

Useful devices:

- annotated examples;
- lesson or article index;
- author credibility;
- progress or curriculum logic;
- code/diagram panels;
- full-width editorial breaks.

Keep reading typography separate from UI labels. Avoid making every article preview an identical rounded card. Rows, covers, marginal notes and shifts in measure can create rhythm.

35. Component library and docs pattern

A component library should show real components and states, not only marketing copy. Make install/copy paths obvious. Demonstrate accessibility, theming, composition and coverage.

A strong page includes:

- visual thesis and target user;
- live component examples;
- state or variant matrix;
- code/install path;
- architecture and dependencies;
- accessibility and license;
- docs navigation.

Docs need search, clear hierarchy, readable code and a stable route from concept to implementation. Do not use animation that slows repeated lookup.

36. Event and conversion pattern

An event page must establish date, location or format immediately. Then explain who it is for, what happens, who participates and how to register.

Cinematic imagery can help, but certainty is the conversion foundation. Use program cards or schedule surfaces where scanning matters, a speaker grid only when speakers drive value, and a sticky action only when it does not obscure content.

Contrast can shift between dark event atmosphere and a light schedule/FAQ section. Keep the shift intentional and carry the same type and accent logic through both.

37. Findings from all 245 dark.design listings

The gallery provided 245 unique “Visit site” URLs after deduplicating repeated Trionn and xAI entries and excluding sponsor/footer links.

Of 245:

- 211 were reachable and inspectable;
- 34 failed or were blocked/unavailable;
- failures included 8 HTTP 404, 6 HTTP 403, 5 HTTP 308 not followed by the audit client, 14 URL/DNS/TLS errors and 1 timeout.

Among inspectable pages:

- grid signals: 129, 61.1%;
- motion signals: 152, 72.0%;
- responsive image markup: 121, 57.3%;
- lazy loading: 111, 52.6%;
- video: 92, 43.6%;
- frame/device/browser signals: 64, 30.3%;
- card naming: 48, 22.7%;
- explicit bento/masonry naming: 5, 2.4%.

Construction classification:

- sectioned: 91;
- editorial/freeform: 76;
- app-shell or JavaScript-heavy: 44.

The dominant system is not repeated bento cards. It is a long page with grid alignment, mixed image ratios, responsive media, selective framing, purposeful motion and density rhythm. Minority patterns such as portrait interruption, local bento and editorial freeform create contrast.

Detection caveats matter. DOM class names undercount patterns with different implementation. CSS backgrounds and generated media can escape ratio measurement. A reachable response can still provide limited static HTML for a client-rendered page. The dataset is strongest when used with representative pixel inspection, not as a substitute for it.

Part IV - Component architecture and the shadcn ecosystem

38. shadcn/ui is open code

The official [shadcn/ui introduction](#) states that it is not a conventional packaged component library. It is open component code and a distribution platform for building a component library. This gives direct modification, composable interfaces, predictable distribution and strong defaults. It also means the product team owns upgrades, accessibility preservation, state design and visual coherence.

Do not copy a component and consider the decision finished. Review semantic structure, accessible names, keyboard and focus behavior, complete states, density, responsive behavior, dependencies, bundle impact, motion, reduced-motion behavior and license. The goal is not to make a product “look shadcn.” It is to use inspectable primitives to build a product-specific system.

39. Primitive to page

Build in layers:

1. semantic foundations;
2. accessible primitives;
3. product components;
4. compositions and workflows;
5. pages.

Do not expose primitives directly everywhere. A product component encodes repeated language, states and behavior. Do not create an abstraction after one use unless the API is already clear.

Keep meaning above mechanism. A `DeploymentStatus` component can use badge and tooltip primitives internally; calling it `GreenPill` freezes appearance rather than intent.

40. State completeness

Every applicable component needs default, hover, focus-visible, pressed, active/selected/open, loading/pending, empty, error, success, disabled, read-only and destructive-confirmation states.

Use more than color for important state. Preserve layout where possible so loading does not shift controls. Disabled elements need an explanation when the reason is not obvious. A failed optimistic action needs rollback or recovery.

State design is where a component becomes a product. The default screenshot is only one frame of the real interface.

41. Buttons and actions

Use button hierarchy to communicate consequence: primary for the page’s next action, secondary for a meaningful alternate, ghost for low-emphasis utility, and destructive for irreversible action.

Do not show several primary buttons in one local decision area. A button label names the action and object when useful. “Save changes” is clearer than “Submit.”

Pressed feedback can use slight scale or surface change, but it must not move nearby layout. High-frequency keyboard actions should respond immediately without decorative delay. Loading buttons preserve width, prevent unsafe repeats and communicate completion or failure.

42. Forms and validation

Use persistent labels, appropriate input types and autocomplete. Placeholder text is an example, not a label.

Validation timing should let the user act. Validate after blur or submit for many fields; immediate validation suits constrained formats only when it does not create noise. Preserve entered values after errors.

An error states what is wrong, where it is, and how to fix it. Group-level errors need a summary and focus strategy. Required and optional conventions should be consistent. Help text precedes errors in reading order when both exist. Success must not rely only on green.

43. Navigation

Navigation reflects information architecture, not visual symmetry. Keep primary routes stable. Use breadcrumbs where hierarchy is deep and back behavior is ambiguous.

Sidebars need clear group labels, active state, collapse behavior and mobile replacement. Tabs switch views within a context; they are not generic navigation links. Segmented controls choose between closely related modes.

Keyboard sequence follows visual and reading order. Active state remains visible without hover. On mobile, preserve high-value actions and do not hide everything behind a menu merely to keep the header sparse.

44. Dialogs, drawers, popovers and tooltips

Choose by task:

- dialog for a focused decision or form that blocks background work;
- drawer/sheet for contextual detail or mobile-friendly secondary flow;
- popover for compact anchored interaction;
- tooltip for brief, non-essential explanation.

Focus enters dialogs and returns to the trigger. Escape closes dismissible overlays. Click-outside must not destroy valuable work. Tooltips cannot contain essential interactive content.

Avoid nested overlays. If a dialog opens a popover that opens another dialog, reconsider the flow. On mobile, a popover may become a sheet when space and touch targets demand it.

45. Tables, lists and data density

Use tables for comparison across consistent columns. Use lists for objects with varied secondary information. Do not force responsive tables into unreadable horizontal compression.

Prioritize columns, hide secondary details behind disclosure, allow controlled horizontal scroll, or transform rows into labeled summaries. Retain selection and bulk actions accessibly.

Use tabular numerals, aligned units and honest precision. Loading skeletons should approximate real row structure, not generic gray bars. Empty states explain what qualifies as an item and how to create or import one.

46. Feedback and perceived performance

Use feedback at the scope of the action: inline for local state, toast for non-blocking global confirmation, banner for persistent system context, dialog for consequential decision, progress for work whose duration matters.

Show immediate acknowledgement. For long work, state what is happening and whether the user can leave. Determinate progress is better when real; fake percentages undermine trust.

Optimistic UI is appropriate when success is likely and rollback is understandable. It needs failure handling and accessible status announcements. A toast that disappears before a screen reader announces it is not useful feedback.

47. Empty, loading and error states

First use, no search results, cleared inbox, restrictive filter, missing permission and service failure are different empty-looking conditions. Each needs different copy and action. Do not celebrate a load failure as a clean empty state.

Loading should preserve layout and indicate what is pending. Skeletons help when structure is known and wait is noticeable. For fast operations, a skeleton flash can feel slower than no placeholder.

Error recovery is a product feature. Preserve context, offer retry where safe, avoid duplicate side effects and provide support when the user cannot fix the problem.

48. Registry and library landscape

The official [shadcn registry directory](#) requires indexed registries to be public, open source and schema-valid. Its live index contained 372 entries at research time, 295 marked healthy by monitoring. This is discovery, not endorsement.

Resource	Best use	Caution
shadcn/ui blocks	app foundations, dashboards, auth	adapt identity and language
Kibo UI	accessible composed product components	review dependencies per item
Motion Primitives	controlled animated primitives	require purpose and reduced-motion path
Animate UI	animated familiar components	avoid motion in high-frequency flows
Magic UI	selective marketing effects	easy to overuse
Aceternity UI	expressive landing moments	avoid stacking signature effects
KokonutUI	polished marketing/product components	inspect source and fit
Spectrum UI	responsive shadcn-based components	verify free scope
React Aria	complex accessible behavior	requires deliberate styling

Health changes. Recheck at project start. Never combine several strongly styled libraries without normalizing typography, color, radius, spacing, motion and API shape.

49. Free template research

Templates are useful for infrastructure, routes, metadata, auth scaffolding and composition examples. They are weak foundations when adopted as final visual product.

Evaluate framework currency, license, semantics, accessibility, states, responsive behavior, image/font strategy, demo removal, token architecture and reproducible build.

A template should save setup time, not determine taste. Replace demo copy and assets immediately. Remove unused routes, effects and dependencies. Do not preserve a generic SaaS sequence when the product needs a different belief path.

50. Product showcase study method

For a product such as Linear, Resend, Raycast, Vercel, Railway or Framer, inspect four layers:

1. brand: type, color, voice, motif;
2. marketing: narrative, proof, ratios, rhythm, CTA;
3. product: density, states, navigation, feedback, shortcuts;
4. technical: responsive behavior, loading, motion, accessibility.

Separate transferable principles from distinctive identity. Resend's public [design page](#) connects brand, design system and marketing guidance. The transferable lesson is organizational coherence, not their palette or cube.

Identify the decision, user benefit and tradeoff. A screenshot collection without analysis becomes trend imitation.

Part V - Motion and micro-interaction

51. Motion decision framework

Before animation, ask:

1. Is the action high-frequency or keyboard initiated? Prefer no motion or near-instant feedback.
2. What does motion explain: continuity, state, hierarchy, feedback or process?
3. Does it improve understanding or perceived response?
4. Can CSS express it cleanly?
5. What is the reduced-motion equivalent?

If there is no clear reason, remove it.

Emil Kowalski's [Great Animations](#) emphasizes natural, fast, purposeful, performant and interruptible motion. The [interfaces.dev](#) cheat sheet adds concrete craft constraints: name exact transition properties, avoid animating high-frequency interaction, cross-fade icon swaps and use transitions for interruptible changes.

52. Timing and easing

Starting ranges:

- direct press/small feedback: 100-180ms;
- popover/menu enter/exit: 150-240ms;
- larger layout change: 220-400ms or a tuned spring;
- grouped entrance stagger: 40-100ms, shorter for dense lists.

Distance, size, frequency and voice alter perception.

Use ease-out for entrances and direct response, ease-in for exits leaving attention, ease-in-out for movement between visible states. Springs suit interruptible physical movement and gesture release. Tune stiffness, damping and mass; avoid arbitrary bounce in serious workflows. Exits are usually slightly faster because the user has decided.

53. CSS transitions and keyframes

Use CSS transitions for hover, press, focus, color, opacity and simple transforms. Specify exact properties. `transition: all` can animate unexpected changes and obscures performance.

Use keyframes for a defined independent sequence: one-time reveal, shimmer, status pulse or staged illustration. Avoid infinite animation unless it communicates live state. Pause loops off screen.

Prefer transform and opacity. Animating dimensions or position can cause layout work; measure when necessary. `will-change` is temporary guidance, not a permanent performance switch.

54. Motion for React

Use Motion for React for coordinated enter/exit, layout projection, shared identity with `layoutId`, gestures, interruption and state-driven orchestration.

Its [layout documentation](#) explains that `layout` can animate position and size changes through transforms while `layoutId` connects matching elements between views.

Set a shared `MotionConfig` policy for reduced motion and common transitions. Avoid wrapping everything in motion components. Verify nested layout, text scaling, radius and interruption. Capability does not guarantee tasteful choreography.

55. Enter, exit and presence

Entrance establishes origin; exit confirms destination. Keep direction consistent. A toast that enters from the bottom and exits sideways creates confusion unless swipe explains it.

Do not animate entire pages on every frequent route. Preserve navigation and shared context. Animate changed content or the object retaining identity.

Group related elements. A heading, body and action can enter as one cluster. Long staggering across lists delays information. Exit animation must not block the next action.

56. Layout and shared elements

Use layout animation when an object keeps identity as position or size changes: selected-tab background, expanding card, reordered list or compact-to-detail view.

A shared-element transition needs a clear object match. The destination should feel like the same object continuing. Keep surrounding layout stable. If text distorts during projection, cross-fade it separately.

For reduced motion, replace travel with opacity or immediate state while preserving identity through content and focus.

57. Gestures and direct manipulation

Gestures need affordance, constraints, feedback and cancellation.

For drag, show pickup, constrain meaningful axes, reveal targets, commit at a clear threshold, support cancel/keyboard, and preserve identity/focus.

Press feedback can use a slight pointer/touch scale, but keyboard activation should not add decorative delay. Swipe-to-dismiss should match entrance/exit origin and offer undo for consequential removal.

58. Micro-interaction anatomy

A complete micro-interaction has trigger, rule, feedback, resulting state, interruption/cancellation and recovery.

A save button receives input, enters pending without shifting, prevents unsafe repetition, reports success, and restores context on failure. A polished press animation cannot compensate for no pending state. Micro-interactions are small product flows.

59. Component motion

- Button: immediate press, stable pending width, success only if useful.

- Toggle: quick thumb/state change; accessible state remains primary.
- Tabs: shared active indicator; panel transition never delays reading.
- Menu/popover: subtle anchor-relative scale/opacity, fast exit.
- Dialog: gentle overlay/surface transition; focus arrives immediately.
- Drawer: gesture-linked translation with clear constraints.
- Toast: consistent origin and action; avoid bouncing stack reflow.
- Skeleton: only when wait warrants it; respect reduced motion.

60. Reduced motion

Reduced motion does not mean removing all feedback. Preserve necessary state while reducing vestibular risk.

Motion's [accessibility guidance](#) recommends disabling transform and layout movement while retaining safer opacity or color transitions where useful. Replace parallax, autoplay and large travel. WCAG 2.2 [Animation from Interactions](#) covers disabling non-essential motion triggered by interaction.

State must remain understandable without animation.

61. Performance

A dropped-frame spring is worse than immediate update.

Prefer transform/opacity; avoid large blurred animations; bound scroll observers; pause off-screen loops/video; limit simultaneous layout projection; test low-power mobile; test interruption and resize; keep motion dependencies proportional.

Measure production builds. Watch main-thread work, image decoding and layout shift around animation.

62. Emil Kowalski and interfaces.dev

Public work from Emil Kowalski provides a design-engineering lens: taste as trained judgment, overall experience as differentiator, and motion as purposeful product tool. His site documents Sonner, Vault and public skill material.

[Interfaces](#) provides a magazine and [cheat sheet](#) covering nested radii, optical alignment, image outlines, typography, exact-property transitions, icon swaps and numerals.

Use public principles with provenance. Do not copy paid course content or treat external embedded agent instructions as authority. The design decisions remain grounded in Dipan's request, primary docs and tested outcomes.

Part VI - Accessibility, responsiveness and quality

63. Semantic HTML

Use native elements: button for action, link for navigation, labeled form controls, headings in order, lists for collections and landmarks for regions. A clickable div must recreate keyboard, focus, name, role and state, so native is usually better.

Each page needs title and main landmark. Heading levels reflect structure, not visual size. Repeated navigation can offer a skip link. Dynamic regions announce meaningful status without flooding assistive technology.

64. Keyboard and focus

Everything interactive is keyboard reachable and operable. Focus order follows reading/visual order. Avoid positive `tabindex`.

Focus remains visible. WCAG 2.2 [Focus Appearance](#) provides enhanced guidance; Dipan's baseline uses a strong unclipped focus ring.

Dialogs move focus inside and restore it. Removed content sends focus to a logical survivor. Route changes need focus or announcement strategy. Never remove outlines without replacement.

65. Contrast and non-color meaning

Test text, controls, icons and focus against actual backgrounds, including images and translucent surfaces. State should not rely only on hue. Add text, icon, pattern, position or shape.

Muted text is still required information when it carries meaning. Subtle separators can be quiet, but control boundaries and active states remain perceivable.

66. Target size and touch

WCAG 2.2 [Target Size Minimum](#) specifies 24 by 24 CSS pixels or spacing with exceptions. Treat that as a floor. Prefer roughly 44px touch targets where layout allows, especially primary actions and mobile icon buttons.

Keep destructive and safe actions separated. Give drag handles room. Test safe-area insets and one-handed reach for persistent actions.

67. Form errors

Labels stay visible. Constraints arrive before they are needed. Errors connect programmatically to fields and summarize multi-error submission.

Preserve data. Move focus to summary or first relevant error. Use live regions carefully. Required state, password rules and examples cannot rely on color or placeholder.

68. Responsive systems

Responsive design keeps content and interaction usable across available space, input, zoom and text size.

Use mobile-first CSS where it clarifies priority, fluid type/spacing with capped extremes, container queries for local adaptation, and content-driven breakpoints.

Test navigation, touch targets, order, crops, tables, overlays, sticky elements and safe areas at each change. Landscape mobile and zoom reveal failures common portrait widths miss.

69. Container queries

Viewport breakpoints cannot describe a component used in sidebar, modal and page. Container queries adapt to local width.

Define meaningful modes: stack action when text becomes cramped, move legend below chart, disclose secondary metadata, change toolbar before collision. Use few coherent thresholds, not many tiny ones.

70. Responsive typography

Use `clamp()` for fluid hierarchy but test real line lengths. Display type must not let one word consume a viewport. Body line height remains generous.

Handle long links/IDs. Avoid fixed-height text containers. At 200% zoom, content reflows without two-dimensional scroll except genuinely two-dimensional material.

71. Images and performance

Provide intrinsic dimensions, responsive source sizes and modern formats. Compress for rendered dimensions while preserving originals.

Do not lazy-load primary proof when it harms LCP. Lazy-load lower media. Use video posters and explicit play where autoplay is unnecessary.

Alt describes useful content/function; decoration uses empty alt. Inspect crop and legibility at each breakpoint. Technical responsiveness does not guarantee visual usefulness.

72. Loading and perceived speed

Acknowledge input immediately and preserve layout. Use skeletons only when wait is noticeable and structure known. Spinners show activity but not shape. Background work states what continues and whether leaving is safe.

Optimism suits likely, reversible actions. Otherwise pending and honest progress may create more trust. Avoid fetching heavy assets users may never need.

73. Visual QA

Render narrow mobile, standard mobile, tablet, desktop and wide desktop; supported themes; 200% zoom; increased text; reduced motion.

Test copy extremes and loading/empty/error/success. Inspect clipping, overlap, baseline, line break, border/radius mismatch, contrast, crop and density rhythm. Contact sheets support comparison but do not replace interaction testing.

74. Technical QA

Check semantics, keyboard/focus, runtime/hydration, links/assets, image dimensions and duplicate bytes, bundle/route weight, layout shift, reduced motion, metadata and reproducible build.

Automate deterministic checks, then visually inspect judgment-dependent outcomes. Green CI is not design approval.

Part VII - Dipan's operating system

75. Project kickoff worksheet

Answer before design:

Product: what is being built, for whom, in what situation, and what changes after success?

Flow: what must the user understand and do, what could stop them, and what honest proof exists?

Content: one-sentence promise, up to three proof points, primary action, optional secondary action, trust/legal needs.

Visual thesis: mood plus mechanism, category grammar, accent/surface direction, image roles/ratios, motion purpose.

Constraints: platforms, input modes, accessibility target, performance budget, assets, free/open-source requirements and publication limits.

76. Reference brief

For each reference record exact URL/date, relevant screen, problem solved, decision, rationale, tradeoff, transferable principle and identity not to copy.

Use several references for a load-bearing pattern. One reference encourages imitation. Compare products in the same category and different categories solving the same interaction problem. Keep the board small and analyzed.

77. Visual thesis

A thesis joins mood and mechanism:

- quiet editorial shell around precise product evidence;
- dark operational surface with one luminous live signal;
- warm maker storefront with hand-drawn proof and plain commerce;
- structured technical journal with diagrams and spacious reading.

Every strong decision should support it. If two incompatible theses remain, choose or separate modes.

78. Page recipe selector

- Product SaaS: promise → overview proof → outcomes → workflows → trust/integrations → action.
- Developer tool: concrete claim/code → first success → architecture → reliability → docs → action.
- Desktop utility: promise/download → app overview → workflow crops → shortcuts/platform/privacy → action.
- Course/editorial: thesis → curriculum/index → annotated proof → author trust → offer.
- Component library/docs: live components → states/coverage → install/code → architecture/accessibility → docs.
- Event: date/location/promise → program → people/proof → logistics/FAQ → registration.

Adapt order to evidence and hesitation. This is selection logic, not a fixed template.

79. Implementation sequence

For the Next.js App Router default:

1. private repo and secret hygiene;
2. routes, metadata and fonts;
3. semantic CSS variables/Tailwind mapping;
4. only required shadcn primitives;
5. product components with state examples;
6. content structure before decorative motion;
7. real imagery and responsive crops;
8. shared motion policy;
9. loading/empty/error/success;
10. accessibility/responsive/performance tests;
11. desktop/mobile evidence;
12. review before public deployment.

Open code transfers maintenance. Record source versions and modifications.

80. Review rubric

Score 0-2:

- **Clarity:** user/job, next action, one claim per section.
- **Coherence:** unified tokens/components, intentional rhythm, thesis alignment.
- **Credibility:** sourced claims, legible proof, honest crops.
- **Interaction:** complete states, keyboard/focus, scoped feedback.
- **Responsive/accessibility:** composed mobile, reduced motion, contrast/targets/names.

Low clarity blocks polish. Record the highest-leverage correction.

81. Anti-patterns

Avoid template before truth, generic gradient plus three cards, page-wide bento, screenshot quotas, unreadable UI, repeated assets, fake plus real UI, unsourced metrics, pill/badge clutter, `transition: all`, long staggers, hover-only meaning, hidden focus, compressed mobile, unexplained disabled states, errors disguised as empty state, visually conflicting library imports, and CI presented as visual completion.

82. Master checklist

Brief

- ☐ User, situation, pain, promise, proof and action.
- ☐ One-sentence thesis.

- ☐ Analyzed references and originality boundary.

Foundations

- ☐ Semantic tokens and themes.
- ☐ Type roles/measures.
- ☐ Limited spacing/container/radius/elevation.
- ☐ Icon and asset licenses.

Page

- ☐ One role per section.
- ☐ Legible proof for important claims.
- ☐ Intentional ratios and density rhythm.
- ☐ Local bento only when useful.
- ☐ No repeated image reference/bytes.
- ☐ No coded duplicate of real proof.

Components/motion

- ☐ Complete states, labels and keyboard.
- ☐ Distinct loading/empty/error/success.
- ☐ Every animation has purpose.
- ☐ Exact properties, interruption and reduced motion.

Accessibility/proof

- ☐ Semantics, focus, contrast and targets.
- ☐ Mobile reorders/re-crops.
- ☐ Zoom, content extremes and critical flows.
- ☐ Desktop/mobile pixels inspected.
- ☐ User-visible artifact checked.

83. Free-resource policy

Free means the specific artifact and license permit the intended use without payment. A site's free landing page does not make its templates free. A gallery is inspiration, not an asset license. A free test pack is not the paid full illustration collection.

For every adopted resource record source, version/date, license, attribution, modification policy, included files, dependencies and upgrade path. Prefer official repositories and documentation.

Current dependable starting points include shadcn/ui, React Aria, Radix Primitives, Motion, Lucide, Phosphor, Tabler, Radix Icons, Remix Icon, Inter, Geist and Plus Jakarta Sans. Registry items require per-item inspection.

84. Dipan's expanded inspiration map

Dipan supplied a broad map spanning resource indexes, curated galleries, awards, motion references, illustration and practitioner projects. These sources are not equivalent.

Product standards: [Linear](#), [Aceternity UI](#), [LaunchVideo](#) and [Jakub Krehel's projects](#). Study implementation, product language and interaction, while separating free/public material from paid products.

Curated galleries: [Lapa Ninja](#), [Landingfolio](#), [Land-book](#), [Curated Design](#), [One Page Love](#), [Landing Love](#) and [Recent Design](#). Use systematic category sampling and follow representative destinations. Galleries reveal what is visible now, not automatically what is durable or usable.

Motion and awards: [MotionSites](#) and [CSS Design Awards](#). These overrepresent expressive showcase work. Extract motion vocabulary and art direction, then test against product frequency, accessibility and performance.

Resource and taste inputs: [F Resources](#), [Falak Gala's bookmarks](#), [Ibelick's curated list](#) and [Notioly](#). These are discovery maps; destination terms and licenses govern use.

The exact phrase `Linear.dev` did not resolve or reveal an official surface during verification. The source map therefore preserves the phrase and uses verified [linear.app](#) as the official product standard unless Dipan identifies another intended destination.

85. Durable principles versus trends

A curated collection can show recurring current choices: oversized type, dark gradients, 3D, scroll choreography, grain, glass, marquee and bento. These are trends until tied to a durable principle.

Durable principles include hierarchy, legibility, honest proof, feedback, continuity, grouping, responsive priority, accessible input and recovery. A trend earns use when it supports one of these and fits product voice.

Research galleries in categories and over time. Record the design choice, problem solved, product cost and accessibility/performance cost. Do not tally effects as if frequency proves quality.

86. Maintaining the Design OS

Update when Dipan gives new feedback, a product reveals a recurring success, a standard/library changes, a resource becomes paid/unmaintained, QA exposes a missing rule, or a rule creates uniform work.

Record why. Current direct instruction overrides older preference. Separate personal taste from accessibility and platform constraints. Keep the master skill compact; deep reasoning belongs in report/references.

Appendix A - Representative chapter

Representative chapter sample: Product imagery is evidence, not decoration

A modern landing page does not become credible because every section contains a dashboard screenshot. It becomes credible when each visual answers a specific question at the moment that question appears.

Dipan's default should be: **type establishes the promise; imagery proves the promise; interaction explains the change**. This keeps the page calm without making it empty, and visual without becoming a gallery of interchangeable cards.

Start by assigning a job to every visual

Before choosing an asset or aspect ratio, name its role:

Role	Question it answers	Best default treatment	Avoid
Product overview	"What is this?"	One legible 16:9 or ultrawide product frame	A tiny full app screenshot with unreadable UI
Feature proof	"Can it do this?"	Focused 3:2 or 4:3 crop around one workflow	Repeating the overview image
Outcome proof	"What changed?"	Before/after, result chart, artifact, or customer evidence	Abstract gradient standing in for evidence
Process explanation	"How does it work?"	Three-step diagram, timeline, or controlled motion	Decorative arrows with no real sequence
Editorial pause	"How should this feel?"	Portrait art, photograph, quote, or spacious type	Another framed dashboard
Comparison	"How are these capabilities different?"	Local bento or compact grid	Turning the entire page into bento

If a visual cannot answer one of these questions, remove it or replace it with type. Empty space is preferable to irrelevant proof.

What the 245-site audit says

The complete dark.design collection audit covered 245 unique gallery listings. Of those, 211 were inspectable and 34 were unavailable, blocked, redirected in a way the audit client did not follow, or failed at DNS/TLS/network level. Failures remain in the dataset rather than disappearing from the denominator.

Across 2,768 images with declared dimensions:

- **48.4% were square-ish** - useful for compact cards, feature clusters, and bounded product moments.
- **26.1% were landscape** - useful for feature proof and workflow storytelling.

- **14.0% were ultrawide** - useful for overviews, dividers, and cinematic product surfaces.
- **11.5% were portrait** - a minority pattern that creates editorial interruption and mobile/device focus.

The construction signals reinforce the same conclusion. Grid alignment appeared on 61.1% of inspectable sites, but explicit bento or masonry naming appeared on only 2.4%. Framed browser/device/mockup treatments appeared on 30.3%. Motion signals were common at 72.0%, yet the visual lesson is not “animate everything.” It is that modern pages use a stable grid, a mixed-ratio cadence, selective framing, and local motion to explain state or depth.

These numbers are structural indicators, not a style prescription. Class names undercount patterns with different naming; declared image dimensions exclude CSS-only backgrounds and some generated media. They are strongest when combined with page inspection and representative examples such as [Railway](#), [V7](#), [Ponder](#), [Mainframe](#), [Resend](#), [Sol Reader](#), and [Procreate Dreams](#).

Dipán's ratio cadence

Use ratio to communicate role, not to make a layout look varied after the fact.

1. **Hero proof: 16:9 or 16:10.** Show the product at a useful scale. Crop peripheral navigation before shrinking the main action.
2. **Focused workflow: 3:2 or 4:3.** Leave enough context to orient the viewer, then make the active operation dominant.
3. **Compact capability: 1:1 or 5:4.** Use when several capabilities must be compared locally. Do not force every feature into this shape.
4. **Editorial or device interruption: 4:5, 5:6, or natural portrait.** Use once or twice to change pacing, not as a default product frame.
5. **Section divider or environment: 2:1 and wider.** Reserve for panorama, atmosphere, or an overview that remains readable at that width.

A page should usually have a dominant ratio, one supporting ratio, and an occasional interruption. Random diversity feels as templated as strict uniformity.

Framing rules

Dipán prefers quiet frames. The frame should clarify the edge of the asset and integrate it with the page, not become the subject.

- Use one outer surface, one hairline border, and one inner radius.
- Keep nested radii concentric. If the outer radius is `r`, the inner radius should reflect `r - inset`, adjusted optically.
- Add a 1px image outline where light imagery disappears into a light surface or dark imagery disappears into a dark one. The [interfaces.dev cheat sheet](#) suggests an inset outline at roughly 8% opacity as a practical starting point.
- Use shadow for separation, not glow. Product screenshots rarely need both a bright border and a large shadow.
- A browser bar is justified only when browser context matters. A device frame is justified only when hardware or mobile use matters.

- Never place a coded imitation of the same product scene beside, above, or below the finished image. The fake scene competes with the proof and makes both feel less trustworthy.

Crop for the claim

A screenshot crop should preserve three things:

1. **Orientation:** enough shell, label, or title to show where the viewer is.
2. **Action:** the control, content, or state change that proves the feature.
3. **Consequence:** the output, completion state, or next step.

Everything else is negotiable. Sidebars, inactive tabs, empty canvas, repeated navigation and distant chrome are the first candidates to remove.

On mobile, do not merely scale a desktop screenshot. Choose one of four adaptations:

- **Re-crop** around the action.
- **Reorder** copy and proof so the claim arrives before detail.
- **Progressively disclose** secondary screenshots behind tabs, a carousel, or a controlled stepper.
- **Substitute** a mobile-native frame or focused result when the desktop overview becomes illegible.

The goal is semantic equivalence, not geometric sameness.

Motion belongs to the change

Motion can explain a workflow better than a static image when the important evidence is a state transition. Use it for:

- an item moving between statuses;
- a command producing an artifact;
- a chart or preview responding to a choice;
- a shared element preserving identity between views;
- progressive disclosure where the spatial relationship matters.

Use CSS transitions for small, interruptible state changes. Name exact properties instead of `transition: all`. Use Motion for React when layout, shared identity, gesture state, coordinated enter/exit, or interruption would otherwise require brittle orchestration. Its `layout` and `layoutId` APIs are specifically designed for layout and shared-element transitions ([Motion layout animation](#)).

Emil Kowalski's [Great Animations](#) argues that motion must feel natural, fast, purposeful, performant and interruptible. This aligns with Dipan's preference for purposeful motion over constant ornament. A section should still communicate its claim when motion is removed.

For people who prefer reduced motion, preserve educational state changes while replacing large transforms, parallax and autoplay with opacity or instantaneous updates. Motion's accessibility guidance recommends disabling transform/layout movement while preserving safer value changes where useful ([Motion accessibility](#)). WCAG 2.2 also requires a mechanism to disable non-essential motion triggered by interaction when it can cause discomfort ([Animation from Interactions](#)).

Implementation pattern

```
<MotionConfig reducedMotion="user">
  <section className="grid items-center gap-10 lg:grid-cols-[0.8fr_1.2fr]">
    <div className="max-w-xl">
      <p className="eyebrow">Planning and progress</p>
      <h2 className="text-balance">A roadmap that stays close to reality.</h2>
      <p className="text-pretty text-muted-foreground">
        Live signals keep progress readable as work changes.
      </p>
    </div>

    <motion.figure
      layout
      transition={{ layout: { type: "spring", stiffness: 320, damping: 32 } }}
      className="overflow-hidden rounded-2xl border bg-card p-2"
    >
      <Image
        src="/proof/roadmap.webp"
        alt="Roadmap showing four initiatives and their current progress"
        width={1500}
        height={1000}
        className="aspect-[3/2] rounded-[calc(var(--radius)-0.5rem)] object-cover"
      />
    </motion.figure>
  </section>
</MotionConfig>
```

The code is intentionally plain. The sophistication comes from the crop, hierarchy, spacing, copy, state and restraint, not from stacking effects.

Review checklist

- ☐ Every visual has a named role and answers a nearby question.
- ☐ No image reference or duplicate image bytes repeat on the same page.
- ☐ No coded mock duplicates the same idea as a real image.
- ☐ The image remains legible at its rendered size.
- ☐ Crops preserve orientation, action and consequence.
- ☐ Ratio follows the visual's job.
- ☐ Dense proof is followed by a quieter section.
- ☐ Framing uses one restrained system.
- ☐ Motion explains change and remains interruptible.
- ☐ Reduced-motion users receive an equivalent, calmer experience.
- ☐ Desktop and mobile are verified from real rendered pixels.

Decision rule

When unsure whether a section needs imagery, ask: **what would the user believe after seeing this that they could not believe from the copy alone?** If the answer is “nothing,” the section does not need an image.

Appendix B - Source ledger

Source ledger - checkpoint 1

Confidence labels: **Primary** = publisher's own documentation/work; **Dataset** = direct audit record; **Reference** = current public directory/curation that requires corroboration.

User and project evidence

- Fifteen local `studies/*/DESIGN-SKILL.md` files - original project-specific skills; 9,211 words total. **Primary internal.**
- Fifteen local `studies/*/reference-notes.md` files, tied to Croct, Raycast, Untitled UI React, Prompt Builder, Woblo, Vite, nan.fyi, UI Design Course, Notioly, Linear, Typefully, Resend, Fumadocs, SVG Guide, and Outskill. **Primary internal / observed references.**
- Dipan's direct instructions in the current WhatsApp thread: minimalist modern SaaS, section-specific composition and ratios, no repeated images, no coded imitation beside real proof, negative space, restrained color, type-led hierarchy, purposeful motion, distinct page identities. **Primary owner evidence.**
- `/memory/knowledge/preferences/build-stack.md`: Next.js App Router, private GitHub, informative landing pages, minimalist/simple themes, hard mobile-responsiveness requirement. **Durable preference lead, corroborated by the current request.**

Full collection dataset

- [dark.design](#) gallery. **Primary collection index.**
- Exact 245 "Visit site" audit in `dark-design-245-audit.csv/json`: 211 inspectable, 34 unavailable; row-level URL/status/reason and coded observations. **Dataset.**
- Representative sources retained in the dataset: [Railway](#), [Framer](#), [V7](#), [Ponder](#), [Mainframe](#), [Rig](#), [Resend](#), [Twingate](#), [Sol Reader](#), [Shape](#), [Opal Composer](#), [Dimension](#), [Procreate Dreams](#). **Primary public sites.**

Component model and free ecosystems

- [shadcn/ui introduction](#): open code, composition, distribution, beautiful defaults; explicitly "not a component library" but a way to build one. **Primary.**
- [shadcn/ui blocks](#): free, open-source application blocks. **Primary.**
- [Magic UI](#): free/open-source animated React component catalog; use selectively, not as a visual identity. **Primary.**
- [Aceternity UI components](#) and [shadcn blocks](#): React/Tailwind/Motion effects and sections. **Primary.**
- [Motion Primitives](#): composable animated interface primitives. **Primary.**
- [Origin UI source](#), [blocks.so](#), and [21st.dev](#): candidates for the larger comparison table; license and current free scope must be checked per item before recommendation. **Primary/reference mix.**

Design engineering, taste, and motion

- [Interfaces](#) and its [cheat sheet](#): design-engineering magazine and concise craft rules for optical alignment, nested radii, image outlines, type, exact-property transitions, interaction details. **Primary.**
- [Emil Kowalski](#), [Great Animations](#), [Developing Taste](#), [AI Skills for Design Engineers](#), and [public skill repository](#). **Primary public work.**
- [Motion for React](#), [layout animation](#), [gestures](#), [accessibility](#), and [useReducedMotion](#). **Primary.**

Accessibility and responsive quality

- [WCAG 2.2](#), [Focus Appearance](#), [Target Size Minimum](#), [Animation from Interactions](#). **Primary standard/explanations.**
- [MDN prefers-reduced-motion](#). **Primary technical reference.**

Typography and icons

- [Inter](#), [Geist](#), [Plus Jakarta Sans](#). **Primary font sources; free license verification retained in final directory.**
- [Lucide](#), [Phosphor](#), [Tabler Icons](#), [Radix Icons](#), [Remix Icon](#). **Primary.**

User-supplied research expansion - September 19, 2026

Exact URLs preserved from Dipan's WhatsApp message: - [F Resources](#) - live index exposed 12 outbound resource links in static HTML, including Recraft, Ibelick backgrounds, Blush, Dark Design, Wonderflow and DrawKit. **Resource index; inspect destinations and licenses individually.** - [Linear.dev](#) - the exact hostname <https://linear.dev/> did not resolve in DNS and returned no search results. The verified official product surface is [Linear](#), which links official customers, pricing, Now, docs, app, status, GitHub and YouTube. **Primary product standard; retain the unresolved phrase caveat.** - [Falak Gala bookmarks](#) - the static response exposes the personal site/navigation but bookmark items are client-rendered; deeper browser inspection required for inventory. **Personal curation.** - [Ibelick's curated-inspiration list](#) - exposed 23 outbound galleries including Minimal Gallery, Dark Mode Design, SaaS Landing Page, Lapa Ninja, Landingfolio, Land-book, ecomm.design, The Responsive and CSS Design Awards. **Practitioner curation.** - [Lapa Ninja](#) - fetch tool identified a 7,300+ landing-page collection; direct request returned a 403 challenge. **Collection, current access caveat.** - [Landingfolio](#) - landing pages, section/component examples, a component library and templates; homepage exposed hundreds of internal links and representative external destinations. **Collection plus mixed free/premium resources.** - [Land-book](#) - gallery title verified; direct request returned 403 challenge. **Collection, current access caveat.** - [CSS Design Awards](#) - nominees/winners with visible judges/about methodology and current external destinations. **Award/gallery; selection favors expressive showcase work, not necessarily product usability.** - [Recent Design](#) - websites, OG images, app-store screenshots, app icons, tools, skills and jobs. **Multi-surface design collection.** - [Curated Design](#) - real live websites, browse and section-level indexes; current homepage exposed 38 external destinations. **Collection with commercial store/pricing layers.** - [One Page Love](#) - one-page inspiration, genre categories, learning material and templates; current homepage exposed representative live sites. **Long-running collection plus commercial templates.** - [Landing Love](#) - stated 2,150 listings, including 1,379 minimal, 566 3D, 562 portfolio and

other categories; also carries commercial placements. **Collection with motion-oriented examples and advertising caveat.** - [Notioly](#) - Notion-style illustration system; current site advertises a paid 500+ pack and a free test pack. **Illustration reference; not a wholly free library, so only the free pack or visual-system principles fit the user's free-resource boundary.** - [LaunchVideo](#) - commercial agent-oriented launch-video product with sign-in, terms, privacy and refunds. **Product/process reference, not a free asset source. Earlier owner decision favors Remotion plus a self-built skill pack rather than purchasing it.** - [Jakub Krehel](#) - verified public projects/writing include interface details, gestures, drag, shared layout animation, gradients, OKLCH, AI as design engineer, shadows, concentric radii, loading.dev, oklch.fyi and interfaces.dev. **Primary practitioner source.** - [Aceternity UI](#) - components, blocks, templates and pricing; free component scope must remain distinct from paid All-Access/templates. **Primary library/product showcase.** - [MotionSites](#) - premium AI website prompts, MCP, animated backgrounds and Academy, with commercial external library links. **Motion/trend reference, not a default free dependency.**

87. Representative gallery sampling: what the current collections actually reward

The added inspiration collections were sampled as systems rather than treated as one undifferentiated moodboard. I inspected their visible indexes, category structures, section indexes, pricing boundaries, and representative destinations where access allowed. Lapa Ninja and Land-book returned access challenges to the research client, so their collection scale and purpose are recorded without pretending that blocked listings were inspected. This distinction matters: a gallery homepage can describe its scope, but only destination sampling reveals whether a repeated treatment is a useful product pattern or merely a fashionable cover image.

Sampling matrix

Collection	Evidence sampled	Repeated signal	Use in Dipan's workflow	Caveat
F Resources	directory structure and outbound resource classes	tools grouped by job, not one visual style	start research by need: type, image, motion, component, proof	destination licenses still govern
Falak Gala bookmarks	practitioner bookmark taxonomy and destinations	high-signal personal curation beats exhaustive feeds	use as a lead list, then verify originals	curation is taste evidence, not permission
Ibelick's inspiration list	23 outbound gallery destinations	meta-curation is useful for finding specialized indexes	select the narrowest gallery for the product question	many destinations overlap or monetize access
Landingfolio	landing pages, section categories, components and templates	concrete section-level comparison supports implementation	compare several solutions to the same section job	free and premium layers are mixed
Curated Design	live-site and section indexes, 38 visible external destinations	production websites reveal hierarchy and proof in context	follow through to live destinations and record behavior	store/pricing layers are commercial
One Page Love	single-page collection and templates	strong compression of narrative into one scroll	study promise-to-proof ordering	template availability and license vary
Landing Love	landing-page and motion-oriented examples	transitions and scroll reveals shape perceived quality	borrow timing vocabulary only after purpose review	expressive motion is overrepresented
Recent Design	recent-site index	a useful view of current visual vocabulary	detect trends, never set standards from recency alone	freshness is not durability
CSS Design Awards	nominees, winners, judging/about material	art direction and technical spectacle are selection signals	extract one device at a time, then usability-test it	award bias favors showcase behavior
Aceternity UI	components, blocks, templates and pricing	effects are packaged as recognizable signatures	use free open components selectively and rewrite identity	free component scope differs from paid products
MotionSites	premium prompts, animated backgrounds and academy	motion is sold as a style layer	use as a trend radar and motion vocabulary	commercial reference, not a free dependency
Notioly	free test pack alongside paid 500+ collection	illustration systems create coherence through constraints	use only the explicitly free pack or study the system	full library is paid

Collection	Evidence sampled	Repeated signal	Use in Dipan's workflow	Caveat
LaunchVideo	product, sign-in, policy and refund surfaces	a repeatable production workflow is the real product	study process and output structure	commercial product, not a free asset
Linear	current official product surface at linear.app	density, keyboard fluency, quiet hierarchy and fast feedback	study product behavior, not only marketing screenshots	supplied Linear.dev hostname did not resolve

Cross-collection findings

The strongest durable pattern is evidence immediately after a precise promise. Across current SaaS-oriented collections, the most reusable pages state one narrow outcome, then show an interface, workflow or artifact large enough to inspect. This survives changes in gradient fashion, radius size and typeface because it answers a stable trust question: “What will I actually use or receive?”

Section indexes are more useful than page feeds during implementation. A page feed mixes industries and narrative jobs. Section-level indexes let a designer compare several navs, pricing tables, testimonials or integration explanations against the same functional requirement. The correct use is not to pick the prettiest card. It is to identify the information each solution makes easy to scan, then rebuild for the current product's content.

Motion galleries systematically over-sample opening spectacle. Cursor followers, cinematic text entrances, scroll-bound transforms, glowing backgrounds and depth-heavy transitions are memorable in gallery thumbnails. They are weak defaults for high-frequency product actions. Dipan's rule should remain: use expressive motion at low-frequency narrative moments; use short, interruptible state transitions in the product; remove both under reduced-motion preferences when continuity can be preserved without travel.

Commercial layering is normal and must be recorded per artifact. Several collections combine free examples, paid templates, subscriptions, courses or full libraries on the same domain. “Found on a free website” is not a license. Every selected asset needs its own source URL, license note and allowed-use record before it enters a product.

Recent visual trends are converging. Current collections frequently surface oversized grotesk headlines, compact pill controls, dark neutral canvases, electric gradients, mesh or noise texture, floating browser frames, monospace metadata, 3D objects, scroll reveals and dense logo rows. These can date quickly when used as the page's identity. Keep them only when the product's content explains their presence.

Distinctiveness comes from constraints, not decoration count. The best reusable lesson from curated destinations is the consistency of a small world: one type contrast, one image behavior, one accent logic, one spatial rhythm, and one motion character. Adding unrelated fashionable devices weakens identity even when every device is polished.

88. Durable principles versus visual trends

Use this classification before a reference influences a build.

Durable principle	Trend expression often confused with it	Test
clear hierarchy	enormous display type	does hierarchy survive at mobile size and in plain text?
visible system state	animated status pill	is the status still clear with animation removed?
continuity between states	shared-layout morph everywhere	does the transition reduce reorientation on this action?
credible proof	tilted browser mockup	can the user inspect a real result or workflow?
grouping by relationship	bento grid	would the grouping remain meaningful in a linear list?
restrained emphasis	neon gradient accent	is only the intended action or idea emphasized?
strong rhythm	alternating card mosaics	do density changes match the narrative jobs?
brand coherence	one fashionable font pairing	are type choices tied to product character and readability?
helpful depth	glassmorphism and glow	does depth communicate stacking, focus or interactivity?
responsive priority	desktop layout scaled down	what was reordered, collapsed or removed for the small screen?

A trend is not forbidden. It must earn its place with product meaning, remain usable without its decorative layer, and avoid becoming the only memorable thing about the interface.

89. Gallery research protocol for future products

1. Write the product question before opening a gallery: hero proof, pricing comparison, dense application shell, onboarding feedback, or another specific job.
2. Choose two broad collections and one specialized source. Do not open ten feeds without a hypothesis.
3. Sample at least five examples for the same job and follow at least three through to their live destination when available.
4. Record the page type, audience, section job, information order, proof mechanism, image role, motion purpose and mobile change.
5. Separate the underlying pattern from its styling. Write both in different fields.
6. Mark whether the source is a gallery, practitioner essay, primary product, open-code component, template, free asset or commercial reference.
7. Verify the exact artifact license. A collection's accessibility does not transfer rights from a destination.

8. Reject references that require invented claims, duplicate imagery, unreadable contrast or motion without reduced alternatives.
9. Synthesize one page thesis from the product's own content. Do not average the references into a generic SaaS page.
10. Revisit the live destination during QA. A captured thumbnail cannot prove responsive behavior, interaction or current availability.